



U1

COMPONENTES BÁSICOS

1. Índice

- 
- [JButton](#)
 - [JLabel](#)
 - [TextField](#)
 - [JCheckBox](#)
 - [JRadioButton](#)
 - [JTextArea](#)

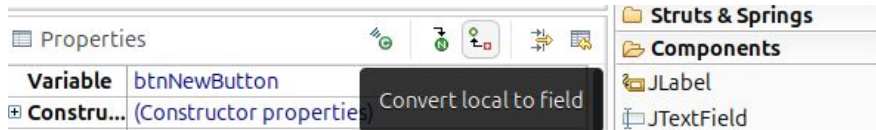
1. Componentes Gráficos Básicos en Aplicaciones de Usuario

Al desarrollar aplicaciones gráficas, cada formulario está compuesto por **componentes gráficos** que permiten la comunicación entre el usuario y la aplicación. Hay muchos tipos de componentes, con distintas utilidades y niveles de complejidad, pero la mayoría de las aplicaciones utilizan siempre **un conjunto básico de componentes**. Nos centraremos en estos componentes básicos, estudiando principalmente **sus usos habituales**.

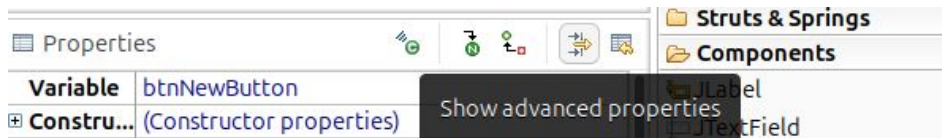
Los componentes básicos que se estudiarán son:

- Botón
- Etiqueta
- Caja de texto
- Casilla de verificación
- Botón de opción
- Área de texto

- Declarar objeto de forma local o global.



- Ver las configuraciones adicionales de los componentes.



1. JButton

Componente Botón (JButton)

Un **botón** es un componente al que se le puede asociar una acción que se ejecutará cuando se haga clic sobre él. En Java, para crear un botón se utiliza la clase `javax.swing.JButton`.

Gráficamente, el botón puede mostrar un **texto**, como “Pulsa este botón”, y al pulsarlo se ejecutará la acción previamente programada.

Propiedades más utilizadas de JButton

- **Background**: color de fondo del botón.
- **Cursor**: tipo de cursor al pasar el ratón sobre el botón.
- **Enabled**: si está activado, el botón es funcional; si no, solo es visible y no interactuable.
- **Focusable**: determina si el botón puede recibir el foco mediante el teclado (Tab).
- **Font**: características de la fuente (tipo, tamaño, estilo).
- **Foreground**: color del texto del botón.
- **HorizontalTextPosition**: posición horizontal del texto respecto a la imagen del botón.
- **Icon**: imagen que se muestra en el botón.
- **IconTextGap**: separación entre el texto y la imagen del botón.

1. JButton (II)

Propiedades más utilizadas de JButton (II):

- **PressedIcon**: imagen que se muestra mientras se pulsa el botón.
- **RollOverEnabled**: si está activado, permite mostrar una imagen al pasar el ratón sobre el botón (RollOverIcon).
- **RollOverIcon**: imagen mostrada al pasar el ratón sobre el botón.
- **SelectedIcon**: imagen mostrada cuando el botón está seleccionado.
- **Text**: texto que aparece en el botón.
- **ToolTipText**: mensaje que aparece al situar el cursor sobre el botón.
- **VerticalTextPosition**: posición vertical del texto respecto a la imagen del botón.

Eventos más utilizados de JButton

- **ActionPerformed**: evento que se dispara al hacer clic sobre el botón.

1. JButton (III) - EJEMPLO



2. JLabel

Cuando necesitamos mostrar un **texto estático** dentro de nuestros formularios.

Normalmente, las etiquetas **no generan eventos**. Su función principal es **mostrar información fija** en los formularios y **acompañar a otros componentes** para indicar su propósito.

También se pueden usar para **mostrar imágenes**. Además de ser informativo, **JLabel permite personalización visual** (colores, fuentes, alineación, etc.) para adaptarse al diseño de la interfaz.

Propiedades más utilizadas de JLabel

- **Border:** Define el borde del componente.
- **Cursor:** Indica la imagen del puntero del ratón al pasar sobre el componente.
- **Enabled:** Si está activada, el componente es funcional; si no, solo se muestra sin permitir interacción.
- **Focusable:** Determina si el componente puede recibir el foco (por ejemplo, con la tecla Tab).
- **Font:** Define el tipo y tamaño de la fuente del texto.
- **Foreground:** Color del texto.
- **HorizontalAlignment:** Alineación horizontal del texto.
- **HorizontalTextPosition:** Posición horizontal del texto respecto a la imagen del componente.

2. JLabel (II)

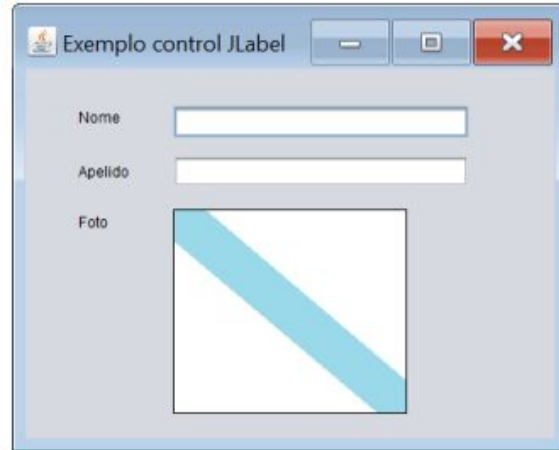
Propiedades más utilizadas de JButton (II):

- **Icon:** Imagen que se muestra en la etiqueta.
- **IconTextGap:** Espacio entre el texto y la imagen.
- **Text:** Texto mostrado en la etiqueta.
- **ToolTipText:** Mensaje emergente (tooltip) que aparece al dejar el ratón sobre el componente.
- **VerticalAlignment:** Alineación vertical del texto.
- **VerticalTextPosition:** Posición vertical del texto respecto a la imagen.

Eventos en JLabel

No es habitual generar eventos a partir de acciones sobre las etiquetas, aunque es posible hacerlo

2. JLabel (III) - EJEMPLO



3. JTextField

El componente **JTextField** se usa para que el usuario introduzca **datos alfanuméricos** en un formulario.

Permite tanto que el usuario escriba texto como que la aplicación muestre texto de forma automática (como una etiqueta).

Normalmente, cada caja de texto va acompañada de:

- Una **etiqueta (JLabel)** que indica qué tipo de dato debe introducirse.
- Un **botón** que, al pulsarse, recoge los datos introducidos y los procesa.

Propiedades más utilizados de JTextField

- **Border:** Define el borde del componente.
- **Cursor:** Indica la imagen del puntero del ratón al pasar sobre el componente.
- **Enabled:** Si está activada, el componente es funcional; si no, solo se muestra sin permitir interacción.
- **Focusable:** Determina si el componente puede recibir el foco (por ejemplo, con la tecla Tab).
- **Font:** Define el tipo y tamaño de la fuente del texto.
- **Foreground:** Color del texto.

3. JTextField (II)

Propiedades más utilizadas de JTextField (II):

- **Icon:** Imagen que se muestra en la etiqueta.
- **IconTextGap:** Espacio entre el texto y la imagen.
- **Text:** Texto mostrado en la etiqueta.
- **ToolTipText:** Mensaje emergente (tooltip) que aparece al dejar el ratón sobre el componente.
- **VerticalAlignment:** Alineación vertical del texto.
- **VerticalTextPosition:** Posición vertical del texto respecto a la imagen.

Eventos en JTextField: [Estos eventos permiten **validar lo que el usuario escribe**]

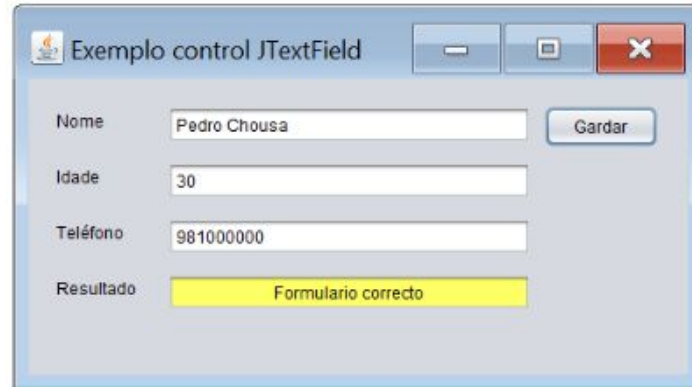
- **KeyPressed:** Al pulsar una tecla. Si se mantiene pulsada, se repite.
- **KeyReleased:** Al soltar una tecla.
- **KeyTyped:** Al escribir un carácter.

```
private void keyTyped(KeyEvent evt) {  
    if (Character.isDigit(evt.getKeyChar())) {  
        evt.consume(); // Evita que se dibuje el carácter  
    }  
}
```

Métodos más utilizados de JTextField:

- **getText():** Recupera el texto contenido en la caja.
- **setText(String text):** Establece el texto de la caja.

3. JTextField (III) - EJEMPLO



The image shows a Java Swing window titled "Exemplo control JTextField". The window contains a form with three text input fields and a "Gardar" button. The first field is labeled "Nome" and contains the text "Pedro Chousa". The second field is labeled "Idade" and contains the number "30". The third field is labeled "Teléfono" and contains the number "981000000". Below these fields is a yellow rectangular area labeled "Resultado" which contains the text "Formulario correcto". The window has standard Mac OS X window controls (minimize, maximize, close) in the top right corner.

Nome	Pedro Chousa
Idade	30
Teléfono	981000000
Resultado	Formulario correcto

4. JCheckBox

El componente **JCheckBox** se utiliza cuando necesitamos recoger **valores lógicos (verdadero/falso o sí/no)** del usuario. También puede emplearse para **mostrar estados** (por ejemplo, si una opción está activa o no).

Consiste en una **pequeña caja que puede estar marcada o desmarcada**, acompañada de un texto que indica su función.

Propiedades más utilizadas de JCheckBox:

- **Background:** Color de fondo del componente.
- **Cursor:** Imagen del puntero al pasar el ratón sobre el componente.
- **Enabled:** Si está activado, el usuario puede interactuar con él.
- **Focusable:** Permite que el componente reciba el foco (por ejemplo, con Tab).
- **Font:** Tipo y tamaño de letra.
- **Foreground:** Color del texto.
- **HorizontalTextPosition:** Posición horizontal del texto respecto al icono.
- **HorizontalAlignment:** Alineación horizontal del texto.
- **HorizontalTextPosition:** Posición horizontal del texto respecto a la imagen del componente.
- **Icon:** Imagen que se muestra junto a la casilla.
- **IconTextGap:** Espacio entre el texto y la imagen.

4. JCheckBox (II)



Propiedades más utilizadas de JCheckBox (II):

- **PressedIcon:** Imagen mostrada al presionar la casilla.
- **RollOverEnabled:** Permite mostrar una imagen diferente al pasar el ratón.
- **RollOverIcon:** Imagen mostrada al pasar el ratón sobre la casilla.
- **Selected:** Indica si la casilla está marcada o no.
- **SelectedIcon:** Imagen mostrada cuando la casilla está marcada.
- **Text:** Texto que aparece junto a la casilla.
- **ToolTipText:** Mensaje emergente (tooltip) al dejar el ratón sobre el componente.
- **VerticalTextPosition:** Posición vertical del texto respecto al icono.

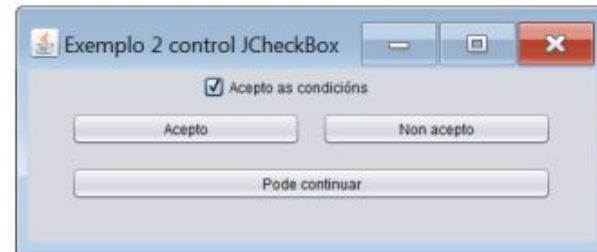
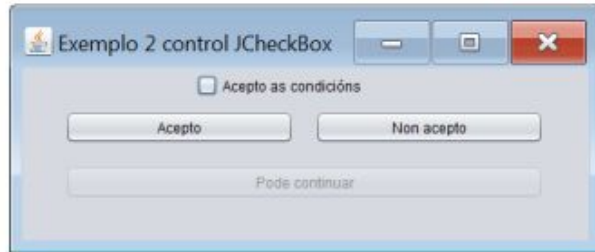
Eventos en JCheckBox:

- **ItemStateChanged:** Cuando cambia el estado de la casilla (se marca o se desmarca).

Métodos más utilizadas de JCheckBox:

- **isSelected():** Devuelve true si la casilla está marcada y false si no lo está.
- **setSelected(boolean b):** Marca (true) o desmarca (false) la casilla.

4. JCheckBox (III) - EJEMPLO



5. JRadioButton

El componente **JRadioButton** se utiliza cuando los datos que pedimos al usuario pueden tener **uno solo entre varios valores posibles**. [A diferencia de las casillas de verificación (**JCheckBox**), **solo puede seleccionarse una opción dentro de un grupo.**]

Estos botones van **agrupados en conjuntos** mediante el componente **ButtonGroup**, para asegurar que únicamente uno esté activo a la vez.

Propiedades más utilizadas de JRadioButton:

- **Background:** Color de fondo del componente.
- **ButtonGroup:** Define a qué grupo pertenece el botón de opción (solo uno del grupo puede estar activo).
- **Cursor:** Imagen del puntero al pasar el ratón.
- **Enabled:** Si está activado, el componente es funcional; si no, solo se muestra.
- **Focusable:** Permite que el botón reciba el foco (por ejemplo, con Tab).
- **Font:** Tipo y tamaño de fuente.
- **Foreground:** Color del texto.
- **HorizontalTextPosition:** Posición del texto respecto a la imagen.
- **Icon:** Imagen que se muestra en el botón.
- **IconTextGap:** Espacio entre el texto y la imagen.
- **PressedIcon:** Imagen mostrada al presionar el botón.

5. JRadioButton (II)

Propiedades más utilizadas de JRadioButton (II):

- **RollOverEnabled:** Permite mostrar otra imagen al pasar el ratón.
- **RollOverIcon:** Imagen mostrada al pasar el ratón sobre el botón.
- **Selected:** Indica si el botón está seleccionado.
- **SelectedIcon:** Imagen mostrada cuando el botón está marcado.
- **Text:** Texto que aparece junto al botón.
- **ToolTipText:** Mensaje emergente (tooltip) al dejar el ratón sobre el componente.
- **VerticalTextPosition:** Posición vertical del texto respecto al icono.

Eventos en JRadioButton:

- **ItemStateChanged:** Cuando cambia el estado del botón (se selecciona o se deselecciona).

Métodos más utilizadas de JRadioButton:

- **isSelected():** Devuelve true si el botón está seleccionado y false si no.
- **setSelected(boolean b):** Marca (true) o desmarca (false) el botón de opción.

5. JRadioButton (III) - EJEMPLO



5. JTextArea



El **JTextArea** es un componente que permite **introducir o mostrar texto multilínea**, a diferencia de **JTextField** que solo muestra una línea.

Está orientado a **texto plano**, con limitada capacidad de formateo.

Propiedades más utilizadas de JTextArea:

- **Background:** Color de fondo.
- **Border:** Borde del componente.
- **Columns:** Número de columnas visibles inicialmente.
- **Cursor:** Imagen del puntero al pasar sobre el componente.
- **DisabledTextColor:** Color del texto cuando el área está deshabilitada.
- **Editable:** Indica si el usuario puede escribir en el área.
- **Enabled:** Si está activado, el usuario puede interactuar con el área.
- **Focusable:** Permite que reciba el foco con Tab.
- **Font:** Tipo y tamaño de letra.
- **Foreground:** Color del texto.
- **LineWrap:** Permite que el texto se ajuste automáticamente al ancho del área.
- **Margin:** Márgenes entre los bordes y el texto.

5. JTextArea (II)



Propiedades más utilizadas de JTextArea:

- **Rows:** Número de filas visibles inicialmente.
- **SelectedTextColor:** Color del texto seleccionado.
- **SelectionColor:** Color de fondo del texto seleccionado.
- **TabSize:** Tamaño de tabulación en caracteres.
- **Text:** Texto que aparece en el área.
- **ToolTipText:** Mensaje emergente al pasar el ratón.
- **WrapStyleWord:** Ajusta palabras completas a la siguiente línea cuando LineWrap está activado.

Eventos en JTextArea:

- **KeyPressed:** Al pulsar una tecla.
- **KeyReleased:** Al soltar una tecla.
- **KeyTyped:** Al escribir un carácter.

Métodos más utilizadas de JTextArea:

- **getText():** Recupera el texto del área de texto.
- **setText(String text):** Establece el texto en el área de texto.

5. JTextArea (III) - EJEMPLO



The screenshot shows a Java Swing window titled "Exemplo control JTextArea". The window contains a form with four fields:

- Nome:** A text field containing "Marta Enes".
- Teléfono:** A text field containing "981111111".
- Observacións:** A text area containing the text "Só disponible martes e xoves" and "Mellor chamar polas tardes de 18.00 a 20.00".
- Resumo:** A text area displaying a summary of the information entered in the other fields, formatted as follows:
NOME: Marta Enes
TELÉFONO: 981111111
OBSERVACIÓNS: Só disponible martes e xoves
Mellor chamar polas tardes de 18.00 a 20.00